

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



ARBI RAMADHAN

109082500114

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Searching adalah proses untuk menemukan suatu nilai atau elemen tertentu di dalam sekumpulan data. Dalam ilmu komputer, pencarian merupakan salah satu operasi fundamental yang paling sering dilakukan dalam pengolahan data. Tujuan utama dari pencarian adalah untuk menentukan apakah suatu nilai (disebut key atau kunci) terdapat dalam kumpulan data, dan jika ditemukan, mengembalikan posisi atau lokasi nilai tersebut.

1. Sequential Search

Sequential Search adalah algoritma pencarian paling sederhana yang bekerja dengan cara memeriksa setiap elemen dalam data secara berurutan dari awal hingga akhir. Algoritma ini tidak memerlukan data dalam keadaan terurut.

2. Binary Search

Pencarian biner adalah algoritma pencarian yang jauh lebih efisien dibandingkan pencarian berurutan, namun memiliki prasyarat utama: data harus sudah terurut (baik membesar maupun mengecil). Algoritma ini bekerja dengan cara membagi ruang pencarian menjadi dua bagian secara berulang.

B. Guided

1. Program Sequential Search

```
package main

import "fmt"

func SequentialSearch(arrBuah [5]string, dataDicari
string) int {
    var idx_found int = -1
    for i := 0; i < len(arrBuah); i-- {
        if arrBuah[i] == dataDicari {
            idx_found = i
            break
        }
    }
    return idx_found
}

func main(){
    var arrBuah [5]string

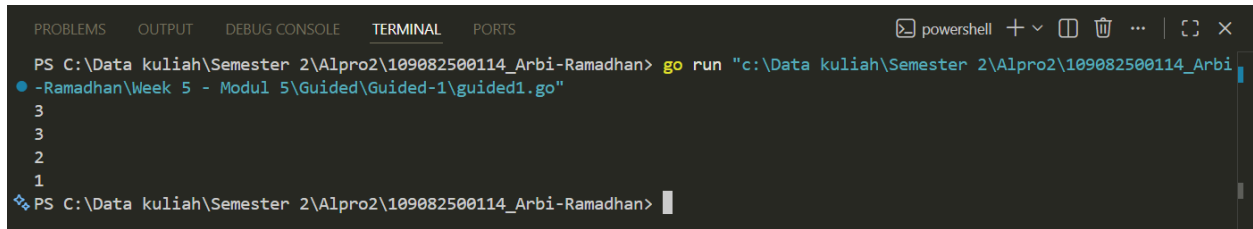
    for i := 0; i < len(arrBuah); i++ {
        fmt.Printf("Masukkan data buah indeks ke-%d :
", i)
        fmt.Scan(&arrBuah[i])
    }

    var dataCari string
    fmt.Println("Masukkan data buah yang ingin dicari :
")
    fmt.Scan(&dataCari)

    var index_data int
    index_data = SequentialSearch(arrBuah, dataCari)

    if index_data < -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-
%d!", dataCari, index_data)
    } else if index_data == -1 {
        fmt.Printf("Data %s tidak ditemukan!",
dataCari)
    }
}
```

`Output :



```
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 5 - Modul 5\Guided\Guided-1\guided1.go"
3
3
2
1
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>
```

2. Program Binary Search

```
package main

import "fmt"

const max = 100

type arr [max]int

func BinarySearch(a arr, n int, X int) bool {
    var kiri int = 0
    var kanan int = n - 1
    var found bool = false
    var tengah int

    //ascending
    for kiri <= kanan && !found {
        tengah = (kiri + kanan) / 2
        if a[tengah] < X {
            kiri = tengah + 1
        } else if a[tengah] > X {
            kanan = tengah - 1
        } else {
            found = true
        }
    }
    return found
}
```

```

    }

    func main() {
        var data arr
        var n, X int

        fmt.Println("Input jumlah data: ")
        fmt.Scan(&n)

        fmt.Println("Input data ascending")
        for i := 0; i <= n; i++ {
            fmt.Scanln(&data[i])
        }

        fmt.Println("Data yang dicari: ")
        fmt.Scan(&X)

        if BinarySearch(data, n, X) {
            fmt.Println("Data ditemukan")
        } else {
            fmt.Println("Data TIDAK ditemukan")
        }
    }
}

```

Output :



```

PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 5 - Modul 5\Guided\Guided-2\guided2.go"
4
10
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>

```

3. Program Pencarian pada Array Bertipe Data Struct

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  int
    IPK  float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari
int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim { //kurang dari
median, pencarian ke kiri
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
//lebih dari median, pencarian ke kanan
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari
int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int
```

```

        fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA
TERURUT ---")
        for i := 0; i < len(mahasiswa06); i++ {
            fmt.Printf("=== Masukkan data mahasiswa 06 pada
indeks ke-%d ===\n", i)
            fmt.Print("Masukkan nama : ")
            fmt.Scan(&mahasiswa06[i].nama)
            fmt.Print("Masukkan NIM : ")
            fmt.Scan(&mahasiswa06[i].nim)
            fmt.Print("Masukkan IPK : ")
            fmt.Scan(&mahasiswa06[i].IPK)
            fmt.Println("-----")
        }
        fmt.Println()

        fmt.Print("Masukkan NIM mahasiswa yang ingin dicari
: ")
        fmt.Scan(&nimDicari)
        fmt.Println()

        var idxHasilCariSeqSearch int
        idxHasilCariSeqSearch =
seqSearchStruct(mahasiswa06, nimDicari)

        fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
        if idxHasilCariSeqSearch > -1 {
            fmt.Printf("Data NIM mahasiswa %d ditemukan
pada array di indeks ke-%d!\n", nimDicari,
idxHasilCariSeqSearch)
            fmt.Println("Berikut Detail Datanya : ")
            fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
            fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
            fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
        } else if idxHasilCariSeqSearch == -1 {
            fmt.Printf("Data NIM mahasiswa %d tidak
ditemukan pada array!", nimDicari)
        }
        fmt.Println()

        var idxHasilCariBinSearch int
        idxHasilCariBinSearch =
binSearchStruct(mahasiswa06, nimDicari)

        fmt.Println("== HASIL BINARY SEARCH ==")
        if idxHasilCariBinSearch > -1 {

```

```
        fmt.Printf("Data NIM mahasiswa %d ditemukan
pada array di indeks ke-%d!\n", nimDicari,
idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak
ditemukan pada array!", nimDicari)
    }
}
```

Output :

[masukkan screenshot output]

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program pilkart yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 1: 1 2: 1 3: 2 7: 1 18: 1 19: 2

Jawaban :

```
package main

import "fmt"

func main() {
    const jumlahCalon = 20
    var suara [21]int
    var totalMasuk, totalSah int
    var input int

    for {
        fmt.Scan(&input)

        if input == 0 {
            break
        }

        totalMasuk++

        if input >= 1 && input <= 20 {
            suara[input]++
            totalSah++
        }
    }

    fmt.Printf("Suara masuk: %d\n", totalMasuk)
    fmt.Printf("Suara sah: %d\n", totalSah)

    for i := 1; i <= jumlahCalon; i++ {
        if suara[i] > 0 {
            fmt.Printf("%d: %d\n", i, suara[i])
        }
    }
}
```

Output :

[masukkan screenshot output]

Penjelasan :

Program ini digunakan untuk menghitung perolehan suara dalam pemilihan ketua RT dengan 20 calon bernomor 1 sampai 20. Program membaca satu baris bilangan bulat hingga menemukan angka 0. Setiap bilangan yang dibaca divalidasi: jika berada dalam rentang 1-20, maka dianggap sebagai suara sah dan dicatat ke dalam array suara pada indeks yang sesuai. Bilangan di luar rentang (negatif atau >20) diabaikan. Setelah input berakhir, program menampilkan jumlah total data yang terbaca, jumlah suara sah, serta perolehan suara setiap calon yang mendapatkan suara (hanya yang memiliki suara >0). Program ini memiliki kompleksitas waktu $O(n)$ di mana n adalah jumlah data yang dibaca, dan memori yang digunakan $O(1)$ karena ukuran array tetap 21 terlepas dari berapa banyak data yang diproses.

2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 Ketua RT: 3 Wakil ketua: 19

Jawaban :

```
package main

import "fmt"

func main() {
    const jumlahCalon = 20
    var suara [21]int
    var totalMasuk, totalSah int
    var input int

    for {
        fmt.Scan(&input)

        if input == 0 {
            break
        }
    }
}
```

```

    }

    totalMasuk++

    if input >= 1 && input <= 20 {
        suara[input]++
        totalSah++
    }
}

fmt.Printf("Suara masuk: %d\n", totalMasuk)
fmt.Printf("Suara sah: %d\n", totalSah)

ketua := 0
wakil := 0

for i := 1; i <= jumlahCalon; i++ {
    if suara[i] > suara[ketua] {
        wakil = ketua
        ketua = i
    } else if suara[i] > suara[wakil] && i != ketua
{
        wakil = i
    }
}

fmt.Printf("Ketua RT: %d\n", ketua)
fmt.Printf("Wakil ketua: %d\n", wakil)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&n)

    if n > 0 {
        tampilkanBarisan(n)
        fmt.Println()
    } else {
        fmt.Println("Masukkan bilangan bulat positif.")
    }
}

```

Output :

[masukkan screenshot output]

Penjelasan :

Program ini merupakan pengembangan dari program penghitungan suara sebelumnya, dengan tambahan fitur untuk menentukan pemenang ketua RT (suara terbanyak) dan wakil ketua RT (suara terbanyak kedua). Program membaca satu baris bilangan bulat hingga menemukan angka 0. Bilangan antara 1-20 dianggap suara sah dan dicatat dalam array suara. Bilangan di luar rentang diabaikan. Setelah semua data diproses, program mencari calon dengan suara tertinggi sebagai ketua, dan calon dengan suara tertinggi kedua sebagai wakil. Jika terdapat calon dengan jumlah suara yang sama, calon dengan nomor lebih kecil diprioritaskan. Program menampilkan jumlah suara masuk, jumlah suara sah, nomor ketua RT, dan nomor wakil ketua RT. Kompleksitas waktu program adalah $O(n + c)$ di mana n adalah jumlah data yang dibaca dan c adalah jumlah calon (20), sehingga sangat efisien.

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan.

Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

```
package main
import "fmt"

const NMAX = 1000000
var data [NMAX]int

func main(){
/* buatlah kode utama yang membaca baris pertama (n dan k). kemudian data
diisi
    oleh prosedur isiArray(n), dan pencarian oleh fungsi posisi(n,k), dan
setelah
    itu output dicetak. */
}
```

```
func isiArray(n int){
/* I.S. terdefinisi integer n, dan sejumlah n data sudah siap pada piranti
masukan.
    F.S. Array data berisi n (<=NMAX) bilangan */
}

func posisi(n, k int) int {
/* mengembalikan posisi k dalam array data dengan n elemen. Posisi dimulai
dari
    posisi 0. Jika tidak ada kembalikan -1 */
}
```

Contoh masukan dan keluaran:

No	Masukan	Keluaran	Penjelasan
1	12 534 1 3 8 16 32 123 323 323 534 543 823 999	8	Data 534 berada pada posisi ke-8 dihitung dari awal data.
2	12 535 1 3 8 16 32 123 323 323 534 543 823 999	TIDAK ADA	

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kiri := 0
    kanan := n - 1

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if data[tengah] == k {
            return tengah
        } else if data[tengah] < k {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }
    return -1
}

func main() {
    var n, k int

    fmt.Scan(&n, &k)
```

```
isiArray(n)

hasil := posisi(n, k)

if hasil == -1 {
    fmt.Println("TIDAK ADA")
} else {
    fmt.Println(hasil)
}
}
```

Output :

[masukkan screenshot output]

Penjelasan :

Program ini digunakan untuk mencari posisi suatu bilangan k dalam kumpulan data integer positif yang sudah terurut membesar. Program membaca dua bilangan pertama yaitu n (jumlah data) dan k (bilangan yang dicari), kemudian membaca n data berikutnya. Pencarian dilakukan menggunakan algoritma binary search karena data sudah terurut, yang memiliki kompleksitas $O(\log n)$. Jika k ditemukan, program mencetak indeksnya (dimulai dari 0); jika tidak ditemukan, program mencetak "TIDAK ADA". Untuk n maksimal 1.000.000, binary search hanya membutuhkan sekitar 20 langkah pencarian, sehingga program berjalan sangat cepat meskipun dengan ukuran data yang besar. Program ini dibuat terstruktur dengan prosedur isiArray untuk mengisi data dan fungsi posisi untuk melakukan pencarian biner.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa terdapat dua algoritma pencarian dengan karakteristik berbeda. Sequential search memeriksa data satu per satu dari awal hingga akhir dan dapat diterapkan pada data yang belum terurut, cocok untuk perhitungan suara pemilihan ketua RT serta penentuan ketua dan wakil ketua. Binary search hanya dapat digunakan pada data yang sudah terurut dengan cara membagi data menjadi dua bagian, sangat efisien untuk mencari posisi nilai k pada dataset berukuran besar hingga 1.000.000 elemen dengan kompleksitas $O(\log n)$. Pemilihan algoritma searching yang tepat sangat bergantung pada kondisi data yang digunakan, termasuk ukuran data dan status keterurutannya.